Name - Soham Samanta

Roll no - 23052272

Batch - CSE

Program - BTech

Final Capstone Project Submission:

(ExcelR & KIIT_Feb26_Full Stack Development with Java Program_B4)

Shoply

Full-Stack E-Commerce Web Application

Technology Stack: React.js · Java Spring Boot · MySQL
Author: Soham Samanta· GitHub: soham29640
Repository: github.com/soham29640/Shoply
Document Version: 1.1 | April 2025

Table of Contents

1. Problem Statement
2. Solution & Features
3. Screenshots
4. Tech Stack
5. Unique Points
6. Future Improvements

1. Problem Statement

Online shopping has become an essential part of modern commerce, yet building a production-quality e-commerce platform from scratch remains a complex engineering challenge. Existing open-source storefronts are often either too heavyweight (requiring extensive infrastructure) or too minimal (missing authentication, cart persistence, and order management). There is a clear need for a clean, well-structured reference implementation that demonstrates how a real-world, full-stack shopping application is designed, built, and deployed.

Specific challenges addressed include:

- **Secure user authentication:** Most tutorials skip JWT-based stateless authentication or BCrypt password hashing. A production-ready implementation is needed.
- **Cart & state management:** Client-side cart state must survive page refreshes and integrate seamlessly with a server-side order workflow.
- **Product discoverability:** Users need instant full-text search, category filtering, and price filtering without reloading the page.
- **Order lifecycle:** Placing an order must atomically persist all line items, shipping details, and payment metadata in a relational database.
- **Tech stack cohesion:** Connecting a React SPA to a Spring Boot REST API with correct CORS policy, JWT handshake, and Axios interceptors requires careful integration.

2. Solution & Features

Shoply is a feature-complete full-stack e-commerce web application that mirrors the core shopping experience of Amazon.com. It is built with React.js on the frontend and Java Spring Boot on the backend, connected via a RESTful JSON API secured with JSON Web Tokens (JWT). The application is designed to be modular, easily extensible, and deployable with a single command per tier.

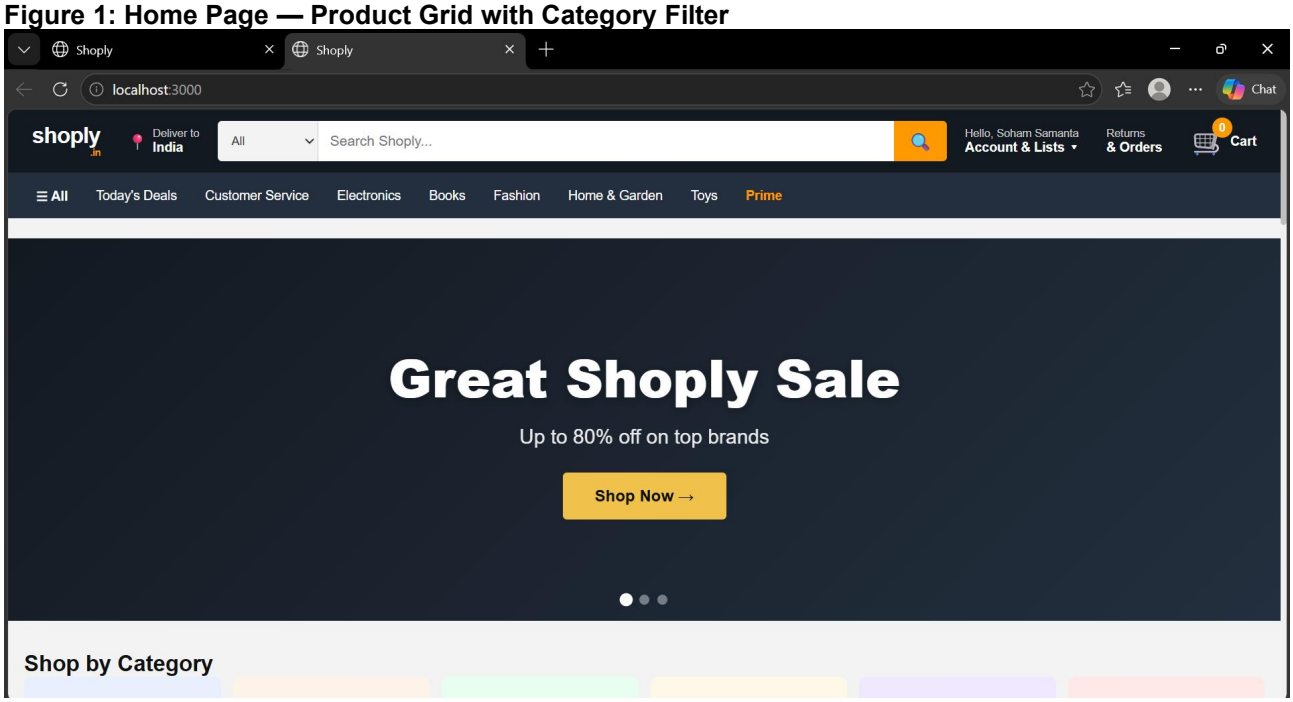
Core Features

- **User Registration & Login:** Secure sign-up and sign-in with BCrypt-hashed passwords and JWT token issuance. Tokens are stored in localStorage and auto-attached to every API request via an Axios interceptor.
- **Product Catalogue:** 12 seeded products across multiple categories (Electronics, Clothing, Books, etc.) with name, description, price, original price (for discount display), brand, rating, stock level, and Prime eligibility flag.
- **Full-Text Search:** Real-time search across product names and descriptions using Spring Data JPA's LIKE query. Results update instantly as the user navigates to /search.
- **Category & Price Filtering:** Users can filter the homepage product grid by category or set a maximum price. Filters are applied server-side for accuracy.
- **Shopping Cart:** Persistent cart management via React Context API with localStorage fallback. Cart item count badge in the Navbar updates live. Backend /api/cart endpoints support server-side cart for authenticated users.
- **Checkout & Order Placement:** A multi-field checkout form collects shipping name, phone, address, and payment method. On submission, a POST /api/orders call persists the order atomically; the cart is then cleared.
- **Order History:** Authenticated users can view all past orders at /orders, with each order showing status, total, items, and shipping details.
- **Admin Product Management:** ADMIN-role users can create, update, and delete products via protected REST endpoints (POST/PUT/DELETE /api/products).
- **Responsive UI:** All pages use CSS Flexbox/Grid and are designed to reflow gracefully on tablet and mobile viewports.

3. Screenshots

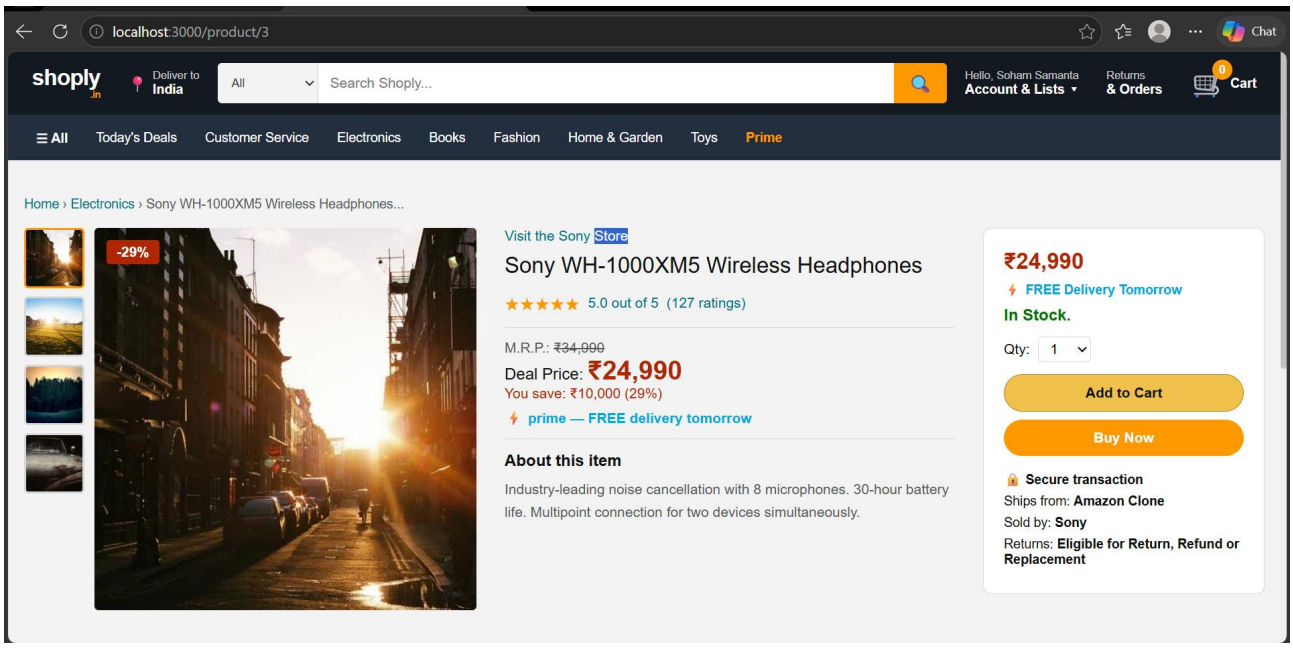
The following screenshots illustrate the main pages and user flows of the application. All UI is rendered by the React.js frontend communicating with the Spring Boot REST API.

Figure 1: Home Page — Product Grid with Category Filter



Displays all products in a responsive card grid. The top navigation bar shows the search box, cart icon with item count, and login/logout button.

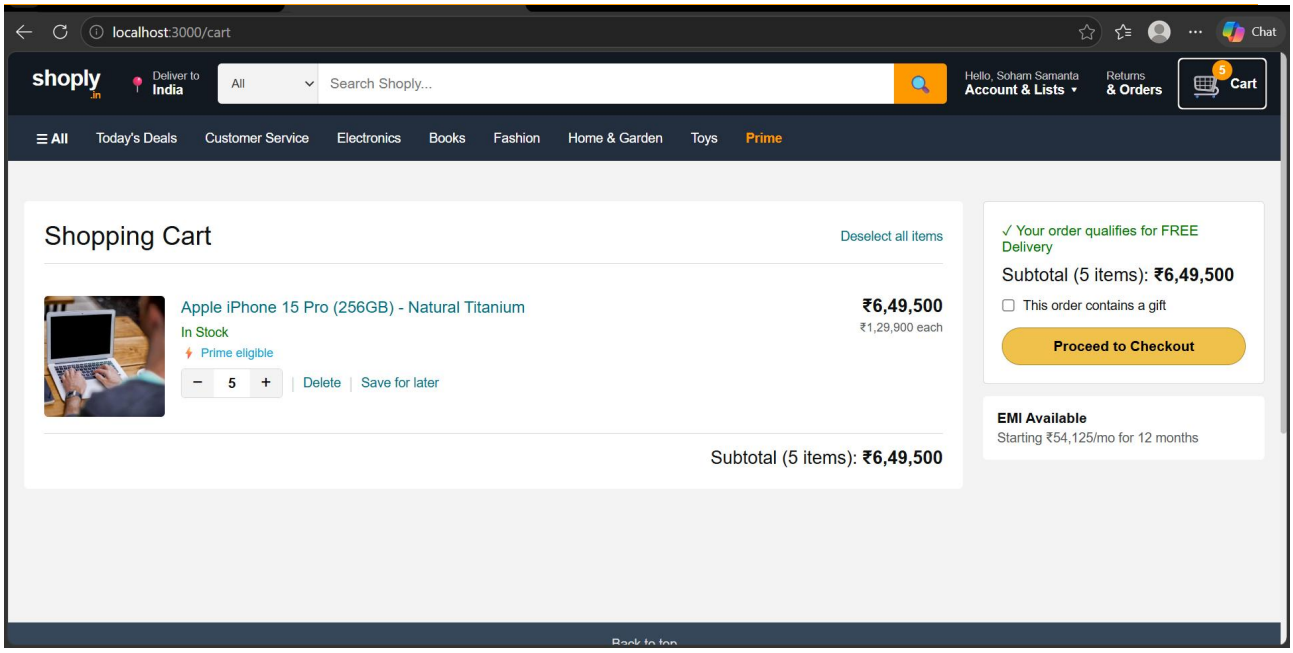
Figure 2: Product Detail Page



Shows full product information: image, description, price, discount %, Prime badge, rating, and the Add-to-Cart button.

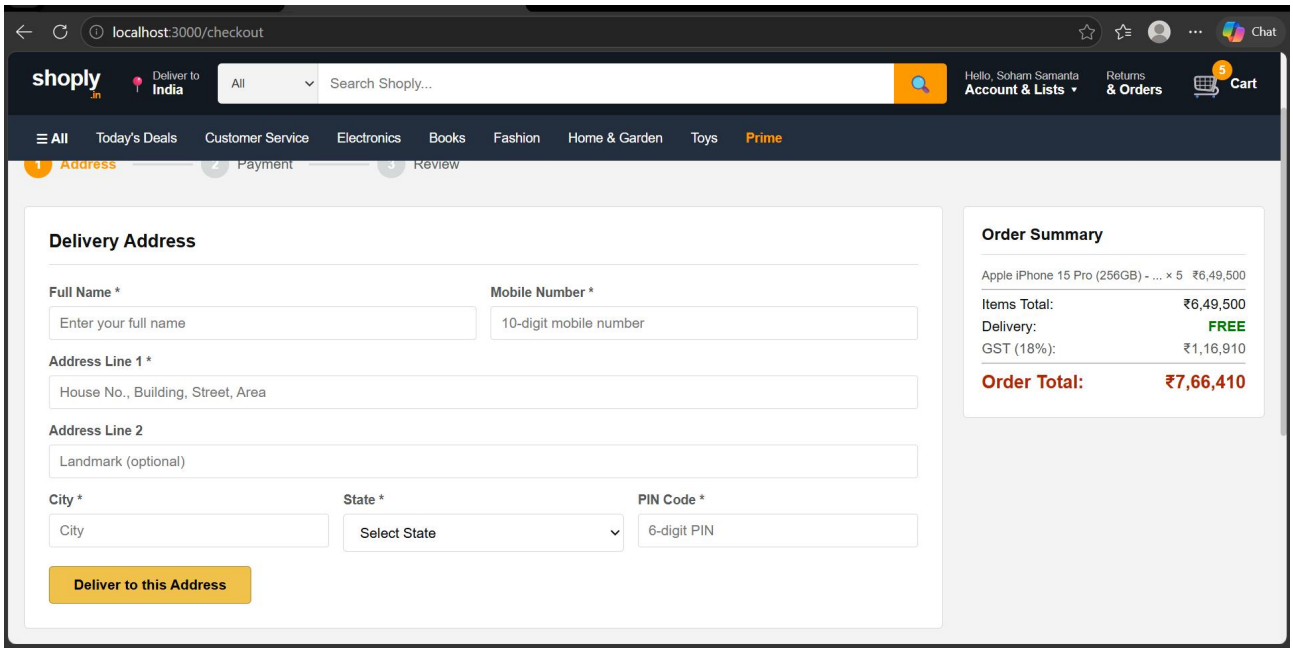
Figure 3: Shopping Cart Page

3. Screenshots



Lists all cart items with product thumbnails, individual and subtotal prices, and quantity adjustment controls. Displays order summary with total.

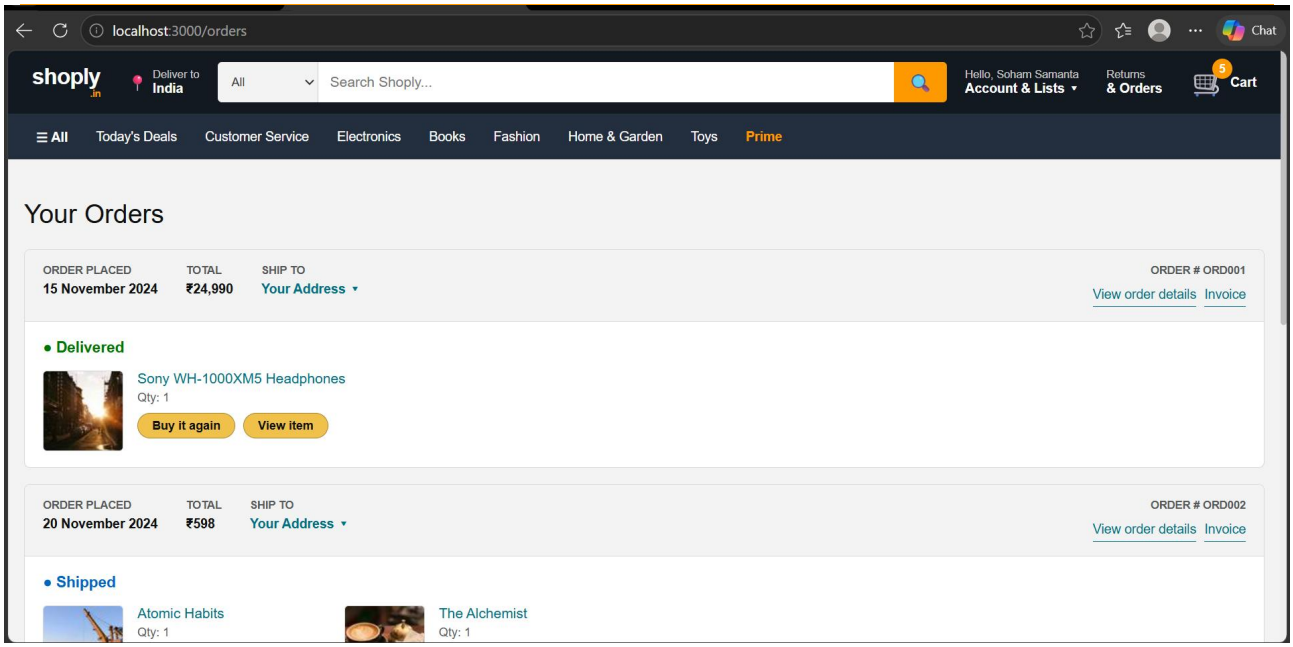
Figure 4: Checkout Page



Multi-field form for shipping address and payment method selection. Submitting places the order via POST /api/orders.

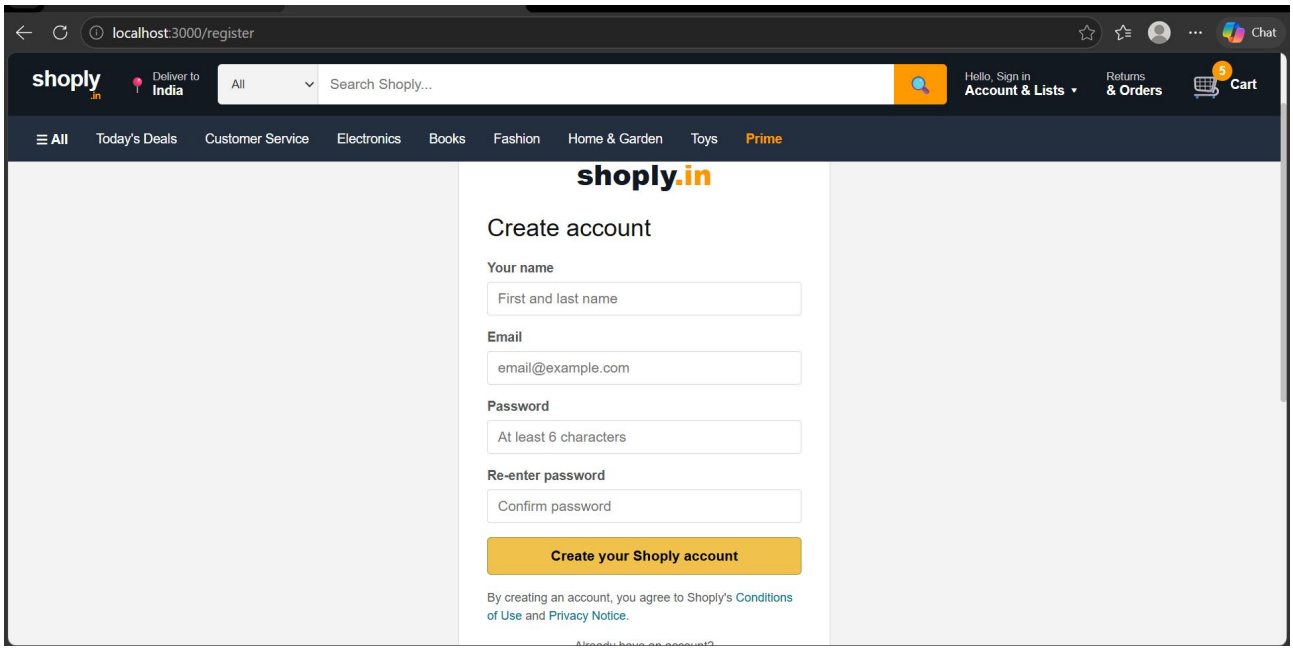
Figure 5: Order History Page

3. Screenshots



Authenticated view listing all past orders with status chip, ordered items, shipping address, and grand total.

Figure 6: Login / Register Pages



Clean authentication forms with field validation and error messaging. Successful login stores JWT and redirects to the home page.

4. Tech Stack

The application follows a standard three-tier architecture: a React single-page application (SPA) as the presentation layer, a Spring Boot REST API as the business logic layer, and MySQL as the data layer.

Layer	Technology	Purpose
Frontend	React.js 18	Component-based UI; Context API for global state
Frontend	React Router v6	Client-side routing (BrowserRouter, Routes, Link)
Frontend	Axios	HTTP client; request interceptor auto-attaches JWT
Frontend	CSS3 / Flexbox	Responsive layout and styling
Backend	Java 17	Core language; records, text blocks, switch expressions
Backend	Spring Boot 3.2	Auto-configuration, embedded Tomcat, bean lifecycle
Backend	Spring Security 6	Filter chain, CORS, route-level access control
Backend	Spring Data JPA	Repository abstraction over Hibernate ORM
Auth	JWT (jjwt 0.11.5)	Stateless token-based authentication
Auth	BCrypt	Password hashing with adaptive cost factor
Database	MySQL 8	Relational RDBMS for persistent data
Build	Maven 3.8	Backend dependency management and build lifecycle
Build	npm 9 / Node 18	Frontend dependency management and dev server
Dev Tools	Lombok	Reduces boilerplate (@Data, @Builder, @Slf4j)
Dev Tools	Spring DevTools	Auto-restart on class change during development

5. Unique Points

The following characteristics distinguish this project from a typical tutorial clone and demonstrate considered engineering decisions:

1. End-to-End JWT Security

Authentication is fully stateless: the Spring Boot backend never stores session data. The JwtAuthFilter validates every protected request independently, making the API horizontally scalable without sticky sessions or a shared session store.

2. MySQL Persistence

The application uses MySQL as the sole database, providing full relational persistence across all restarts. Hibernate's ddl-auto=update ensures the schema evolves safely with entity changes without losing existing data.

3. Context API + localStorage Hybrid Cart

Rather than making a network round-trip on every cart interaction, cart state lives in React Context and is synchronised to localStorage. This provides instant UI feedback and survives full page refreshes without any additional API calls.

4. Layered Architecture Separation

The backend strictly follows Controller → Service → Repository layering. No business logic leaks into controllers; no database queries appear in services. This makes unit-testing each layer independently straightforward.

5. Seed Data via data.sql

12 realistic products with images, prices, categories, ratings, and Prime flags are loaded automatically at startup via data.sql. New contributors can run the project and see a meaningful product catalogue without any manual setup.

6. Role-Based Admin API

Product creation, update, and deletion are restricted to the ADMIN role at the Spring Security layer — not merely by convention or client-side logic. Attempting these endpoints without an ADMIN JWT returns HTTP 403 Forbidden.

7. Axios Request Interceptor

A single, centrally maintained Axios interceptor in utils/api.js attaches the JWT header to every outgoing request. Feature components never handle authentication headers directly, keeping them clean and reusable.

8. Prime Eligibility Flag

Products carry an is_prime boolean, enabling future integration of Prime shipping badges and filtered Prime-only product listings — both already served by the GET /api/products/prime endpoint.

6. Future Improvements

The following enhancements are planned to evolve the project towards a more production-ready, feature-rich platform:

- **Payment Gateway Integration:** Integrate Stripe or Razorpay for real credit/debit card and UPI payments, replacing the current 'Cash on Delivery / Card on Delivery' placeholders.
- **Product Reviews & Ratings UI:** Although the Review entity and ReviewRepository already exist in the backend, the frontend lacks a review submission form and rating histogram. Completing this feature will close the loop on the product feedback lifecycle.
- **Real-Time Order Tracking:** Add WebSocket (STOMP over SockJS) support to push order-status updates (Placed → Shipped → Delivered) to the browser in real time.
- **Admin Dashboard:** Build a dedicated /admin panel with charts (orders per day, revenue, top products), product management CRUD forms, and user management — accessible only to ADMIN-role accounts.
- **Image Upload via Cloud Storage:** Replace static image_url strings with a file-upload workflow backed by AWS S3 or Cloudinary, enabling merchants to upload product photos directly from the admin panel.
- **Wishlist / Saved Items:** Allow authenticated users to save products to a Wishlist, persisted server-side, with one-click 'Move to Cart' action.
- **Email Notifications:** Send transactional emails (order confirmation, shipping update) using Spring Boot Mail (JavaMailSender) with an HTML template rendered by Thymeleaf.
- **Containerisation & CI/CD:** Add a Dockerfile and docker-compose.yml for one-command local startup of both tiers plus MySQL. Integrate a GitHub Actions workflow for automated build, test, and push to Docker Hub on every merge to main.
- **Pagination & Infinite Scroll:** Implement server-side pagination (Spring Data Pageable) on the product endpoints and pair it with an infinite-scroll component in the React frontend to support large product catalogues efficiently.
- **Accessibility & Internationalisation:** Audit all pages against WCAG 2.1 AA, add ARIA labels, and introduce i18next for multi-language support (English, Hindi, Spanish as a starting point).

This document was prepared as part of the Shoply project submission. All diagrams and placeholder images are for illustration; actual screenshots may vary.